

Python

Introduction:

Python was created by Guido van Rossum, and released in 1991.

Python is a programming language and it is open source. It supports multiple programming paradigms including object oriented, functional and procedural programming.

It is also platform independent. python is a top down approach oriented language. It is case sensitive

Datatypes:

Numeric datatype:

Int,float,complex

```
int_data=2
```

```
print(int_data)
```

output: 2

```
float_data=2.3
```

```
print(float_data)
```

output: 2.3

```
complex_data=5+8j
```

```
print(complex_data)
```

output:5+8j

Sequence datatype:

List,tuple,str,range

```
string_data='abirami'
```

```
print(string_data)
```

output:abirami

```
list_data=[1,2,3.7,5+3j,'sha']
```

```
print(list_data)
```

```
output: [1, 2, 3.7, (5+3j), 'sha']
```

```
tuple_data=(6,9.0,6+1j,'vasan')
```

```
print(tuple_data)
```

```
output: (6, 9.0, (6+1j), 'vasan')
```

```
range_data=range(5,10)
```

```
print(range_data)
```

```
output: range(5, 10)
```

set datatype:

set, frozenset

```
set_data={1,4.9,7+2j,'good'}
```

```
print(set_data)
```

```
output: {1, 4.9, (7+2j), 'good'}
```

```
frozenset_data=frozenset([1,2,3])
```

```
print(frozenset_data)
```

```
output: frozenset({1, 2, 3})
```

Mapping datatype:

Dictionary

```
dictionary_data={'id':1,'name':'abi','age':21}
```

```
print(dictionary_data)
```

```
output: {'id': 1, 'name': 'abi', 'age': 21}
```

Operators:

Arithmetic operators:

+, -, *, /, **, //, %

```
num1=10
```

```
num2=5
```

```
print(num1+num2)
```

```
output:15
```

```
print(num1-num2)
```

```
output:5
```

```
print(num1*num2)
```

```
output:50
```

```
print(num1/num2)
```

```
output:2.0
```

```
print(num1**num2)
```

```
output:100000
```

```
print(num1//num2)
```

```
output:2
```

```
print(num1%num2)
```

```
output:0
```

self assignment operators:

$+=, -=, *=, /=$

```
num1+=5
```

```
print(num1)
```

```
output:15
```

```
num2-=2
```

```
print(num2)
```

```
output:3
```

```
num1/=10
```

```
print(num1)
```

```
output:1.5
```

```
num2*=2
```

```
print(num2)
```

output:6

Relational/comparsion operator:

>,<,>=,<=,==,!=

```
print(num1>num2)
```

output:True

```
print(num1<num2)
```

output:False

```
print(num1>=num2)
```

output:True

```
print(num1<=num2)
```

output:False

```
print(num1==num2)
```

output:False

```
print(num1!=num2)
```

output:True

Membership operators:

```
ex="python is a fast growing language"
```

```
print("fast" in ex)
```

output:True

```
print("fast" not in ex)
```

output:False

Identity operators:

```
a=[1,3,5]
```

```
b=[1,3,5]
```

```
c=a
```

```
print(a==b)
```

output:True

print(a is b)

output:False

print(a is c)

output:True

Datatype:

List methods:

Append: (add single element at a time at the end)

```
list_element=[1,5.6,'Python',9.3,3+8j,'passing']
```

```
list_element.append('welcome')
```

```
print('after appling append:',list_element)
```

output: after appling append: [1, 5.6, 'Python', 9.3, (3+8j), 'passing', 'welcome']

insert(add single element at a time)

```
list_element=[1,5.6,'Python',9.3,3+8j,'passing']
```

```
list_element.insert(2,6+4j)
```

```
print('after applying insert:',list_element)
```

output:after applying insert: [1, 5.6, (6+4j), 'Python', 9.3, (3+8j), 'passing', 'welcome']

extend:(add multiple at a time)

```
list_element.extend([1,'two',3.4])
```

```
print('after applying extend:',list_element)
```

output:after applying extend: [1, 5.6, (6+4j), 'Python', 9.3, (3+8j), 'passing', 'welcome', 1, 'two', 3.4]

pop(this will delete the element at the end)

```
list_element.pop()
```

```
print("after applying pop",list_element)
```

output:after applying pop [1, 5.6, (6+4j), 'Python', 9.3, (3+8j), 'passing', 'welcome', 1, 'two']

```
list_element.pop(6)
```

```
print('after appling pop',list_element)
```

output:after appling pop [1, 5.6, (6+4j), 'Python', 9.3, (3+8j), 'welcome', 1, 'two']

remove:(this the element is known the use this)

```
list_element.remove(3+8j)
```

```
print('after appling remove',list_element)
```

output:after appling remove [1, 5.6, (6+4j), 'Python', 9.3, 'welcome', 1, 'two']

clear:(this will delete all the list)

```
list_element.clear()
```

```
print('after appling clear',list_element)
```

output:after appling clear []

update:(as there is no keyword update. We update using the index)

```
list_element[0]=13
```

```
print('after updating:',list_element)
```

output:after updating: [13, 5.6, (6+4j), 'Python', 9.3, 'welcome', 1, 'two']

reversing;

```
list_element.reverse()
```

```
print('after reversing:',list_element)
```

output:after reversing: ['two', 1, 'welcome', 9.3, 'Python', (6+4j), 5.6, 13]

count:#will show occourence of the elmet

```
print(list_element.count(5.6))
```

output:1

getting index of element

```
print('getting index of element:',list_element.index(6+4j))
```

output:getting index of element: 5

sorting: #sort need only homogenous ele that is only numbers

```
unsorted_element=[7,90,-65,38,-1,44]
```

```
unsorted_element.sort()
```

```
print('after sorting',unsorted_element)
```

output: after sorting [-65, -1, 7, 38, 44, 90]

```
unsorted_element.sort(reverse=True)#for descending to asecndeing
```

```
print('after sorting',unsorted_element)
output:after sorting [90, 44, 38, 7, -1, -65]

copy:
copied_ele=unsorted_element.copy()
print('after coping:',copied_ele)
output:after coping: [90, 44, 38, 7, -1, -65]
```

Tuple methods:

Count:

```
tuple_elemnt=(1,5.5,'hello',7+2j,'sql',5.5)
print(tuple_elemnt.count(5.5))
output:2
```

index:

```
print('getting index of element:',tuple_elemnt.index(5.5))
output:getting index of element: 1
```

Set methods:

```
set_elment={1,2.3,5,'hiii',35,5}
```

adding:

```
set_elment.add('language')
print('after adding',set_elment)
```

output:after adding {1, 2.3, 35, 5, 'hiii', 'language'}#as set is unordered the index of element will not be same

pop:# this will delete any value. We cant specify the index

```
set_elment.pop()
print('after popping',set_elment)
output:after popping {2.3, 35, 5, 'language', 'hiii'}
```

remove:#in this we will specify the element

```
set_elment.remove(35)
```

```
print('after removing',set_elment)
```

output: after removing {2.3, 5, 'language', 'hiiii'}

discard:

```
set_elment.discard(35)
```

```
print('after discarding',set_elment)
```

output:after discarding {2.3, 'hiiii', 5, 'language'}

set_elment.discard(3)#this specified is not avabile it will just print the value or ignore it

```
print('after discarding',set_elment)
```

output:after discarding {1, 2.3, 35, 5, 'hiiii'}